

コンテキスト/ハーネスエンジニアリングの現在

Hirosato Gamo

※解釈しやすいよう抽象度の高い表現をしている箇所や個人的な見解を含みます。Microsoftサービスについての正確な情報は公式ドキュメントをご参照ください。

HIROSATO GAMO



➤ Microsoft AI Cloud Solution Architect

LLM隆盛の黎明期からAzure AIを通じたLLM企業導入の技術支援を推進。

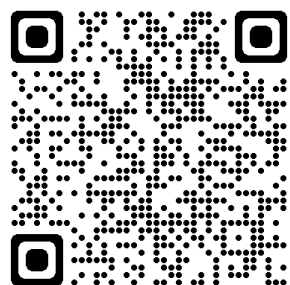
➤ Microsoft Evangelist

SNS上での技術情報の発信や登壇活動に従事。「ChatGPT - Azure OpenAI大全」などの資料が10万ビューを超え「2023 Most Viewed Deck 25」にランクイン。

[2023 - Most Viewed Decks \(speakerdeck.com\)](https://speakerdeck.com/hiro-gamo/2023-most-viewed-decks)

➤ 上智大学大学院 応用データサイエンス学位プログラム LLM概論担当 非常勤講師

➤ 著書「[LLMの原理、RAG・エージェント開発から読み解く コンテキストエンジニアリング](#)」
共著「[Azure OpenAI ServiceではじめるChatGPT/LLMシステム構築入門](#)」

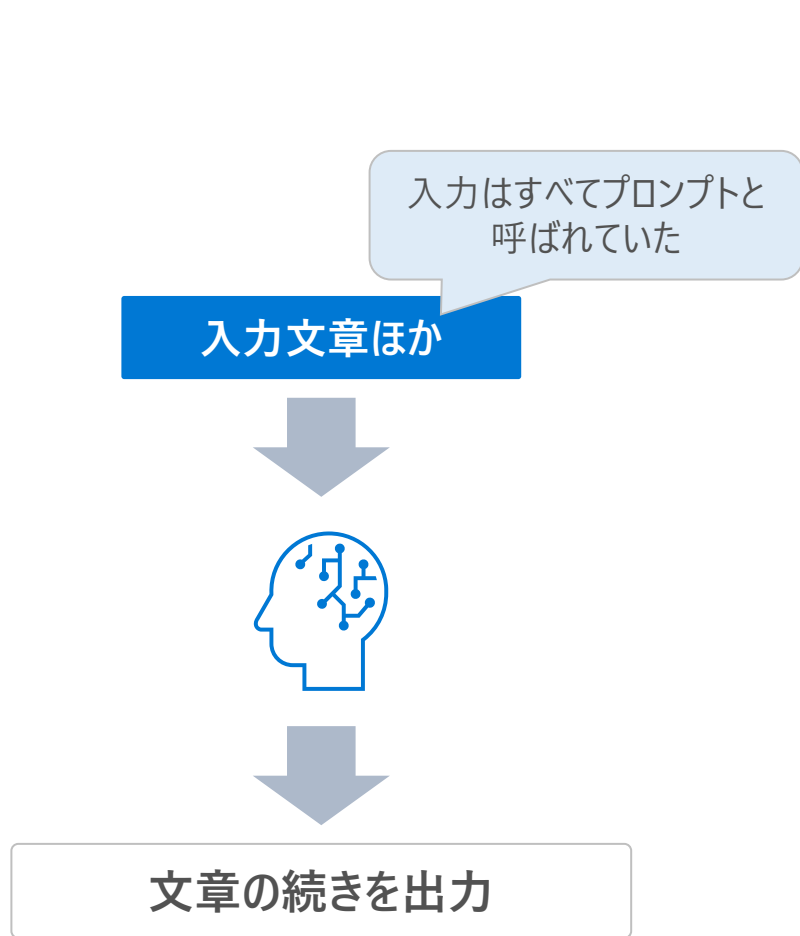


 @hiro_gamo

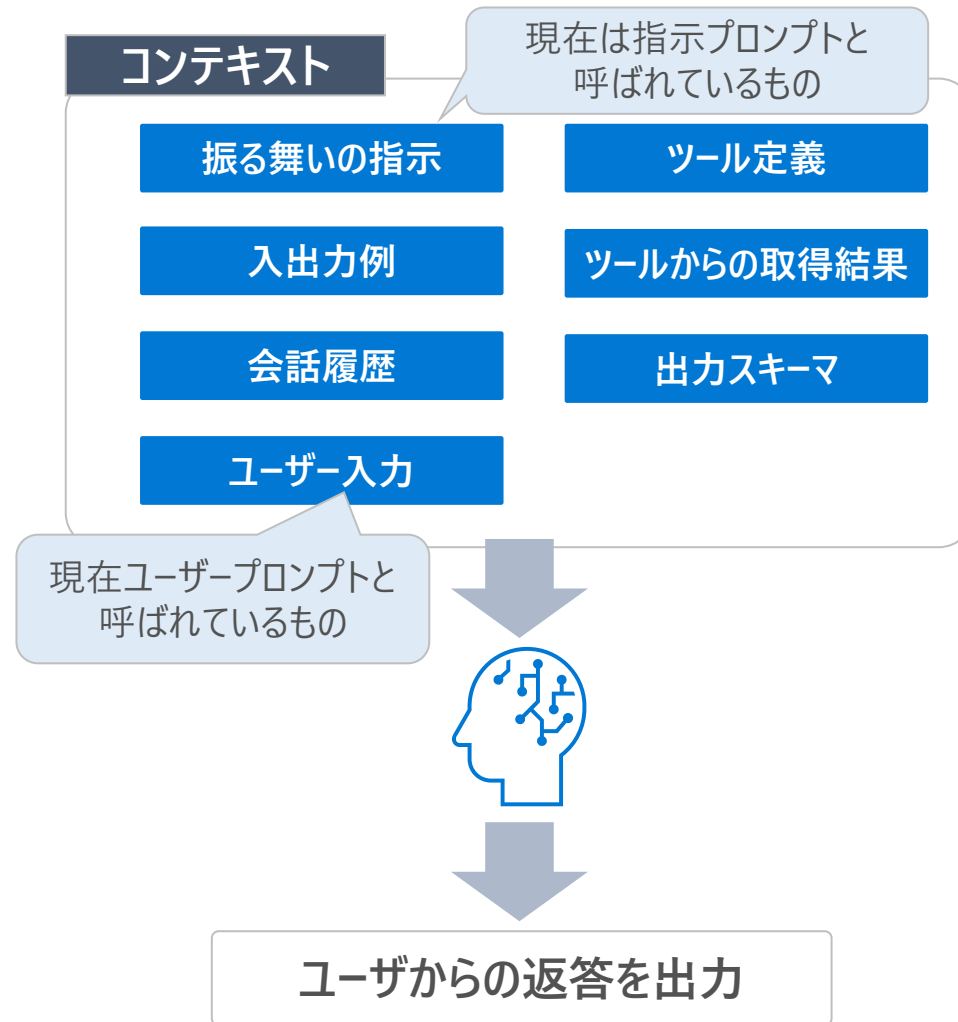


[マイクロソフト エバンジェリスト](#)

LLMにおけるプロンプトとコンテキスト



初期のLLMの入力



現代のLLMへの入力

コンテキストを取り巻く 3つの問題

精度劣化

複雑化による指示不履行

コンテキスト量が増えると、ルールの遵守や適切なツール選択の精度が低下。

- ❗ 設計時のコンテキストの与え方が不適切だと、高性能モデルでも常に隣合わせ。

容量の制約

トークン上限と理解度

入力可能な文字数（トークン）には上限があり、長文入力には限界がある。

- ❗ 上限内であっても、コンテキストが長大になるとモデルの理解度の低下リスクあり。

コスト・速度

非機能面への影響

処理するテキスト量に応じて課金が増大し、応答時間も遅延。

- ❗ リアルタイム性が求められるアプリでは、コンテキストの肥大化は特に致命的。

コンテキストのハンドリングが極めて重要になりつつある

LLMアプリ開発においては、適切なコンテキストを与えつつ、増大する情報をいかに抑制・取捨選択するかが重要。これがユーザー体験に最も影響を与える。

コンテキストエンジニアリングはLLMを推論側(入力)で制御する技術の総称

LLMが最も質の高い回答を返すために、限られた入力領域において、**何を与え何を捨てどのように良いコンディションを保つか**。
この技術の総体が「コンテキストエンジニアリング」。

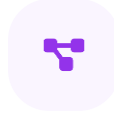
👑 コンテキストエンジニアリングを制すものが、LLMによる未来実現を制す。

📁 構成する7つの具体的要素



コンテキストを受け取るUI/UXの工夫

ユーザー意図を正確に捉え、構造化データとして渡すための設計



コンテキストの分割

ワークフロー化、Agents as Toolsなどによるタスク分解



LLM Inferenceプリセットの整備

精度を確保し健全に動作させるための事前設定のバランスング

振る舞い指示

出力スキーマ

パラメータ

参照データ

例示 (Few-shot)

ツール定義



コンテキストの動的取得

Skillsなどを活用したオンデマンド情報取得



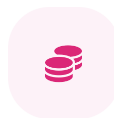
コンテキストの圧縮・削除

ウィンドウの枯渇防止、制度維持のための情報量制御



RAGにおけるクエリ生成、インデックス整備

外部知識を適切に検索・注入するための基盤構築



コストを最適に保つキャッシュ維持

コンテキストキャッシュ機能によるコストとレイテンシの最適化

コンテキストエンジニアリングは誰にとって必要か

LLMアプリケーションのタスク性能はコンテキストエンジニアリングの質に左右される
同じモデルでも、**コンテキスト設計次第でアウトプットの完成度は劇的に変化。**



LLMアプリ開発者

今後あらゆるアプリにLLMが組み込まれていく中で、必須となる基盤技術スキル。



Coding Agentユーザ

AIエージェントの精度と効率を最大化するために、ユーザ側にも文脈設計が求められる。



一般のLLMアプリユーザ(短期的には)

本来は意識せず使えるのが理想だが、現状では市場がそこまでユーザフレンドリでないため、コンテキストの性質を理解しておくことで活用の幅が広がる。

コンテキストエンジニアリングは一過性の技術なのか

「性能が上がれば、すべてをコンテキストに詰め込むだけで済む」と思われがちですが、**3つの構造的な課題**がそれを阻んでいます。



ウィンドウ拡大の停滞

モデルの推論性能の進化スピードに比べ、コンテキストウィンドウサイズの拡大ペースは鈍化傾向に。



タスクの複雑化が先行

解決すべきタスクが高度化しており、必要なコンテキスト情報の増加量がウィンドウ・解釈性の進化速度を上回っている状態。



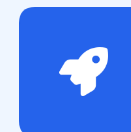
解釈のクセや脆弱性

LLMはノイズに弱く、情報の配置や構成（コンテキスト内の順序など）の独特なクセが出力品質に影響。



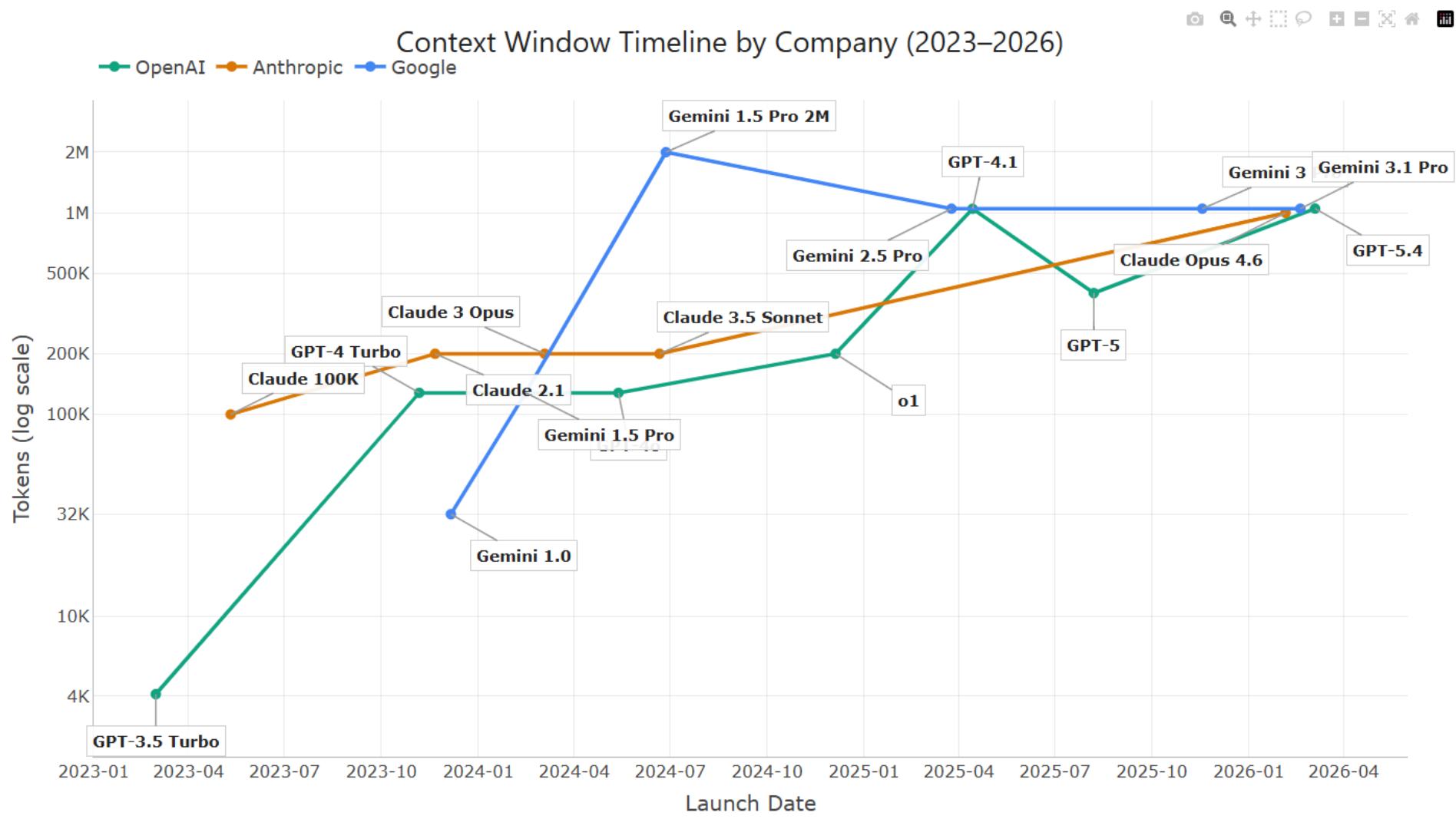
Conclusion

この技術が不要になるまでには、まだ相当の時間がかかる。
当面の間、コンテキストエンジニアリングは実務での競争優位を左右する重要なスキルであり続ける。

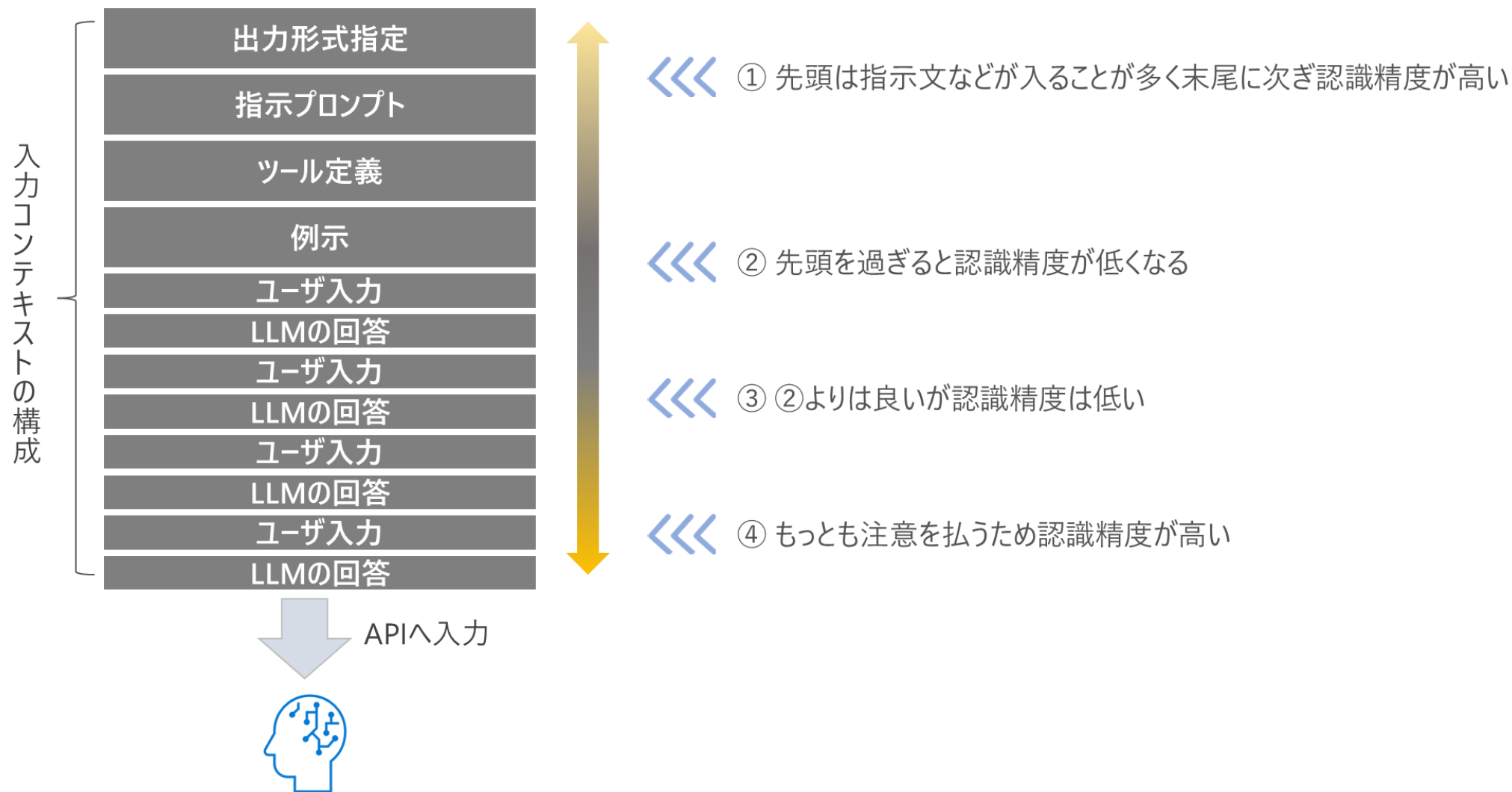


コンテキストウィンドウの進化はやや停滞気味

更にコンテキストウィンドウ限界まで入力した場合、顕著に解釈精度が悪化することが知られている。

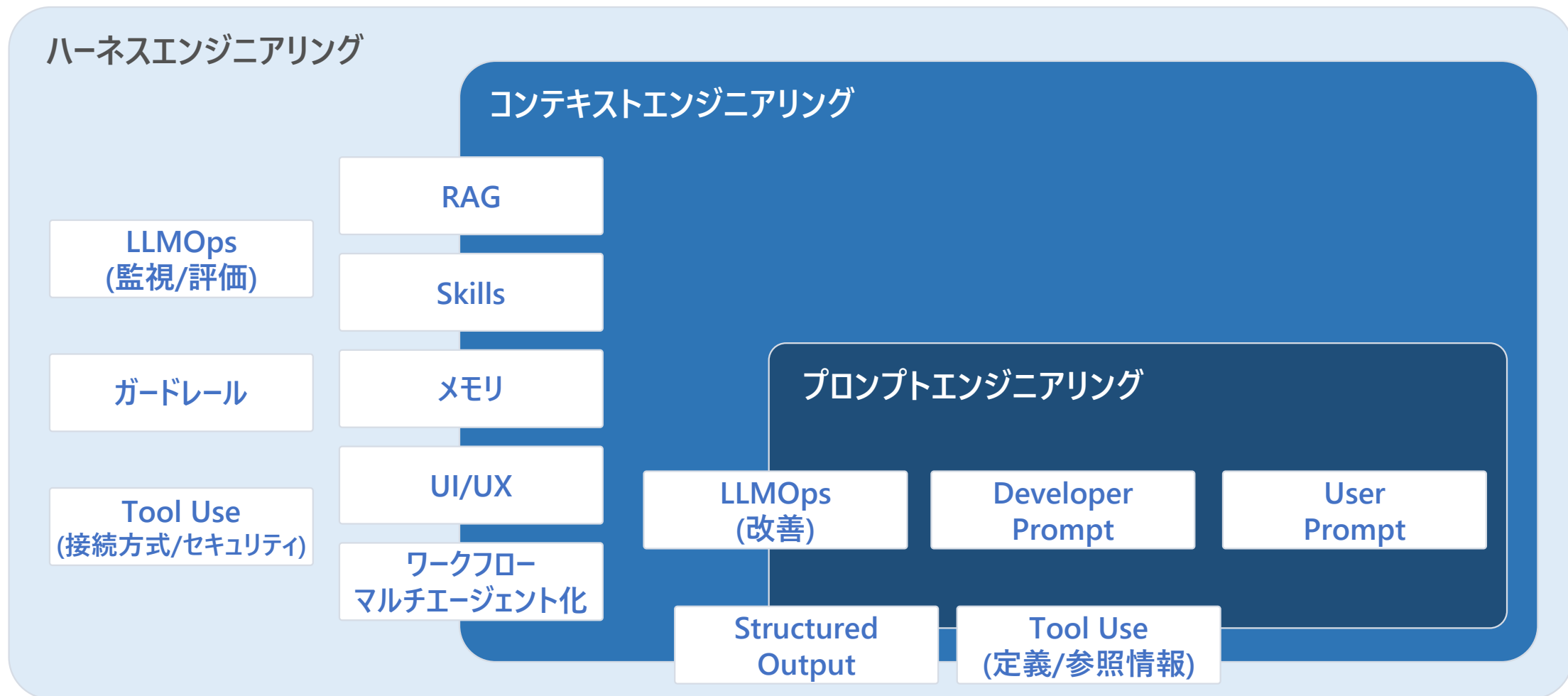


LLMのコンテキスト解釈のクセ「Lost in the Middle」



ハーネスエンジニアリングってなに？ (個人的な整理)

コンテキストの更に外側にある仕組みがハーネスとして意識されるようになりつつある。





コンテキストエンジニアリングの各技術のポイント

チャットを超え、改めて「コンテキストの入り口」UI/UXに目を向ける

👉 UIは「飾り」ではなく、コンテキスト処理の全ての入り口。



1. 起動の最適化

チャットに拘らず、必要に応じ
定型UIや自動トリガー
を選ぶ。



2. 入力の最小化

指示をシステムに寄せ、ユーザ
入力は「**迷わない最小限**」に
留める。



3. 時間の活用

待機時間 = 思考時間。この設計
が**LLMの回答性能を決める**。



4. 人の介入

適切なタイミングで
承認やコンテキスト取得のため
の**人間の介入**を入れる。



5. 状況の可視化

ログ・状態で
「**今何が起きているか**」示す。



6. 次のアクション

一発完了は稀なため、**修正・再
生成**を前提にする。

プロンプトだけではない、推論リクエストにおけるプリセット



Prompt Engineering の本質は1つ。「あらゆる場所で CoT を」

テクニックとしてではなく、**重要な生成の直前に質の良い情報が来るよう**に常にコントロールする。

LLM自身の出力の活用 (Reasoning)

知識生成

LLM内部に持っている知識や論理を中間出力することで
関連情報をコンテキスト化し回答精度を高める。
(推論モデルはオートでこれに外部検索も組み合わせられる)

再帰修正

一度出力した内容を再修正することで、初手での誤りを効率的に検出し
最終回答としては質の良いものに仕上げる。

指示のRecall

指示内容のニュアンスをOutputのフォーマットに組み込むことで
追従性を維持する。

【例】JSON出力でプロパティ名を手掛かりにするCoT

出力JSON



```
{
  "id": "12345",
  "user_impression": 4,
  "short_text": "2023年のMVPは大谷翔平選手。",
  "short_text_in_en": "Shohei Ohtani was the MVP in 2023.",
  "category": [
    {"category_label": "野球", "category_description": "~~~~~"},
    {"category_label": "野球", "category_description": "~~~~~"},
    ...
  ]
}
```

➤ 出力の長さや言語などの指定をプロパティ名に入れ込むことで指示を忘れにくい。



RAG におけるコンテキストエンジニアリング

RAGを始める前に...3つの選択肢



CAG

Context Augmented Generation

概要

外部DBを持たず、プロンプトのコンテキスト内にナレッジを常駐させる手法。キャッシュ技術の発展とLLMのロングコンテキスト解釈力の向上により利用が加速。

✓ メリット

- 実装が非常に手軽
- レイテンシで有利になりやすい

❗ デメリット

- コンテキスト圧迫による性能低下



Built-in RAG

Service Integrated

概要

WorkIQなど、特定のサービスが組み込みのRAG機能を持っている場合、そのAPIをツールとして利用する手法。

✓ メリット

- RAGシステムを構成する必要が無い

❗ デメリット

- 非機能要件がサービスに依存



ユーザマネージド

Custom Built RAG

概要

独自にRAGシステムを構成してチューニングを行う、フルスクラッチに近いアプローチ。

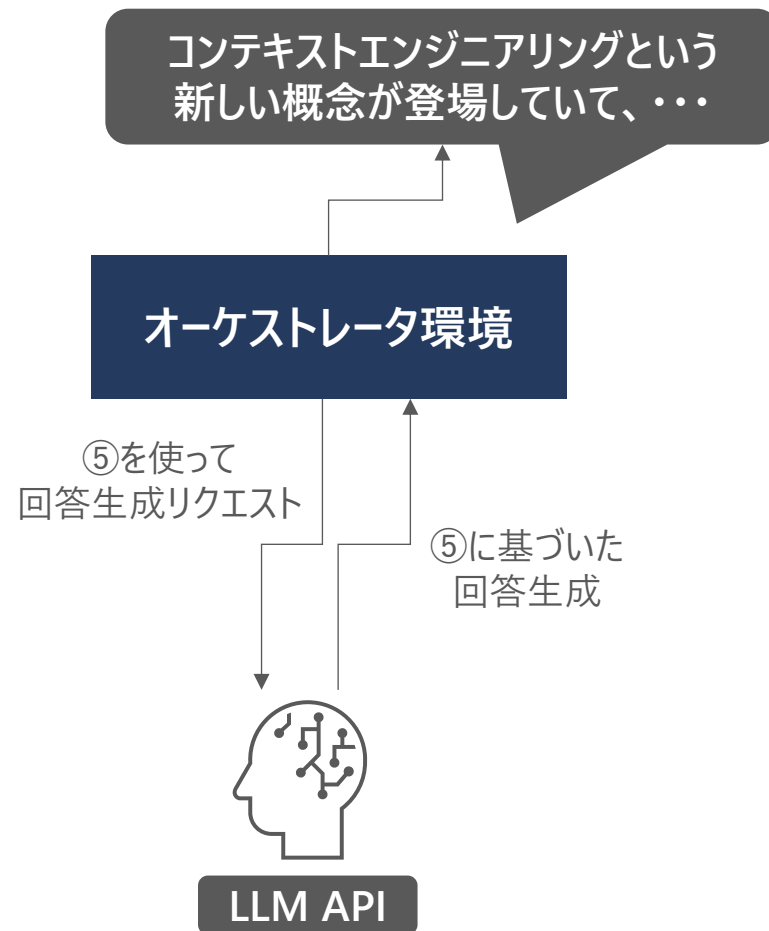
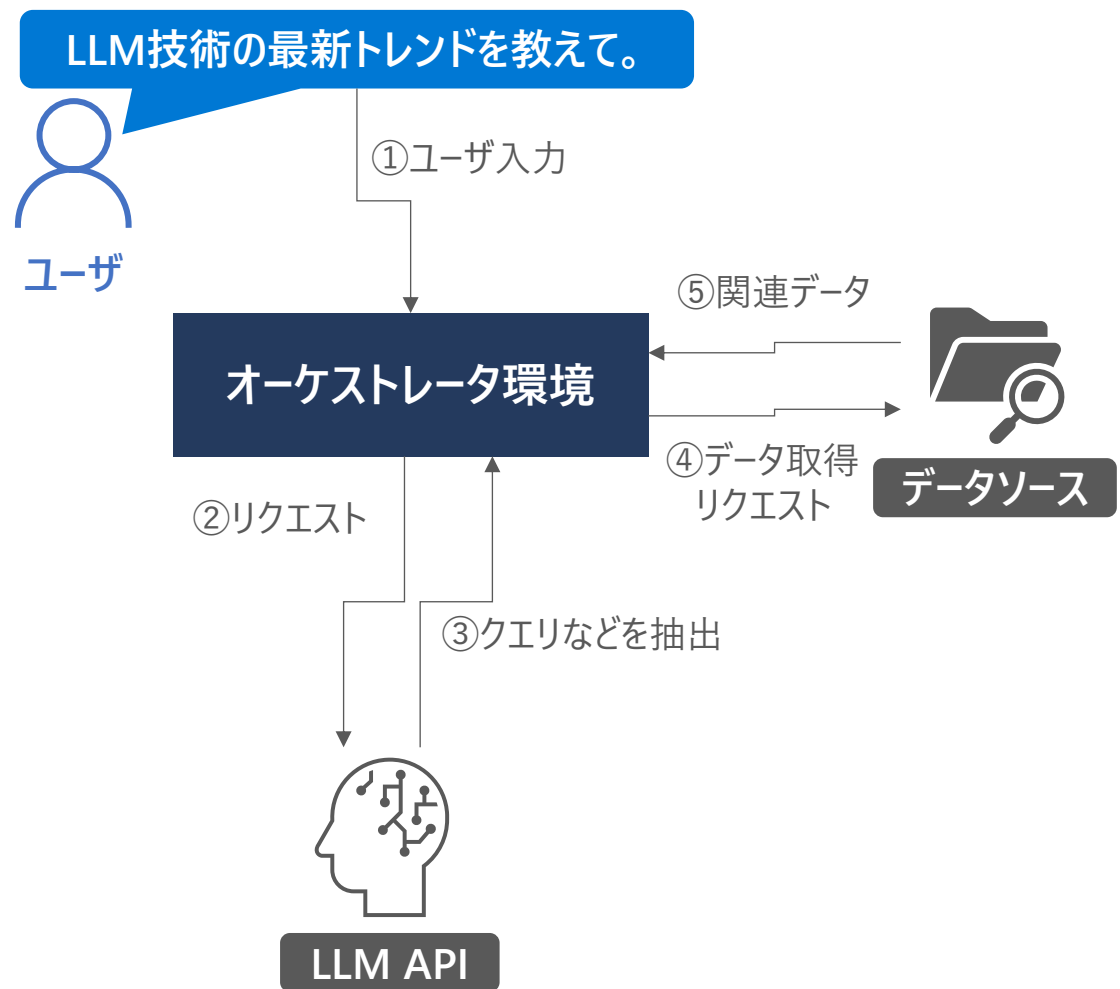
✓ メリット

- 最もカスタマイズ性が高い

❗ デメリット

- 専門知識と中長期の調整管理が必要

改めてRAGの仕組みを振り返る



RAGにおけるデータソースの多様化

現在においては、データソースや取得手段はベクトル検索に限定されず、様々なデータソースを用いる。

データソース名	概要
 ドキュメントなど	社内に存在するPDF、PowerPoint、Excelファイルなどにおけるテキスト情報を抽出しておき、LLMのリクエストに応じて検索。 全文検索エンジンやベクトルストアを使うことが多い。
 Web検索	Web検索APIを実行し、ヒットしたWebページから情報を抽出し回答に利用。 最近はLLMのAPIに組み込みツールとして実装済み。
 データベース	RDB（リレーショナルデータベース）やNoSQL DBをはじめとするデータベースへクエリを実行。取得した数値やレコードデータをLLMの回答生成に利用。
 その他	他にも、関連した情報や事実関係を取得するためにナレッジグラフからの情報取得や、レコメンドエンジンと組み合わせた情報取得など、多様なシステムとの連携がある。

RAG関連技術に起こった数年の発展



精度向上のためのテクニック一覧

RAGにはコンテキストを含む様々な対処が存在。

		施策	概要	備考・トレードオフ
1	インデックス作成	不要なドキュメントの排除	古いファイル、使用頻度の低いファイルの削除	事前のアクセスログ分析が必要
2		検索対象テキスト選択	チャンクに重要なキーワードが欠落しないよう前後情報の要約を足したり、そもそもの検索対象をチャンクに対する想定質問文にするなど、クエリからヒットしやすい形式にする。	LLMによる加工が入った場合、元のドキュメントから情報が欠落する可能性が0ではない。
3		図表情報の適切な抽出	図表をLLMが読み取りやすい形式でテキスト化する。	LLMによる加工が入った場合、元のドキュメントから情報が欠落する可能性が0ではない。
4		Embeddingモデル・類似度関数調整	専門用語に強いEmbeddingモデルに変更したり、類似度計算の学習をして想定に近い検索対象がヒットしやすいようにする。	モデルの動作環境の準備や調整にやや専門性と手間が必要。
5	対話	クエリ加工	クエリにLLM内部の情報を追加したり、検索対象テキストに近くなるような加工を施す。	リッチな加工を施すと回答までに時間が掛かりUXが悪化する。
6		ユーザからの情報収集	検索に入る前に必要な情報をユーザから収集する。	毎回質問を重ねられるとUXが悪いためバランス調整が難しい。
7	検索	ハイブリッド検索の導入	検索エンジンにおけるハイブリッド検索を使用する。	フルテキスト検索の精度が悪いとベクトル検索単体より精度劣化する。
8		リランクの導入	検索エンジンにおけるリランク機能を使用するか、リランクモデルを導入し検索結果を解析させる。	回答までの時間が増加しUXが悪化する。
9		フィルタ検索	インデックス作成時にあらかじめドキュメントのカテゴリを付与しておき、検索実行時にユーザ質問からカテゴリを推定させ、そのカテゴリ内だけのフィルタ検索を実行する。	明白にジャンルが違うドキュメントが混在するケースでないと機能しにくい。
10	回答	結果取り込み件数調整	検索された結果の上位を何件までLLMに渡し参照させるか調整する。	件数を多くし過ぎるとLLMの解釈性が低下し、回答までの時間も増加する。

クエリ拡張・加工の手段

検索エンジンの仕組みとマッチング対象データを把握しながら、適切なクエリ生成を狙う。

HyDE

Hypothetical Document Embeddings



問いに対する仮想的な応答をLLMで生成。(関連用語の生成がされることを期待)
その応答をEmbeddingでベクトル化して文書を検索。

[LangChain](#) でより高い vector 検索精度が期待できる HyDE 仮説をやってみる

クエリ修正



タイポの修正による精度向上が報告されている。またはクエリは質問文で投げられるため、インデックス情報に近い形式に変換することで精度向上が見込める。

[Dealing with Typos for BERT-based Passage Retrieval and Ranking - ACL Anthology](#)

質問分解



単一の質問だけでは解決できない問いに対して、質問を複数に分割する。
検索エンジン側で機能提供されているケースもある。

[Measuring and Narrowing the Compositionality Gap in Language Models | OpenReview](#)

Step Back



詳細な質問に対して、そのままクエリを投げるのではなく、上位概念に一度変換するクエリを発行する。
例えば「大谷翔平の2023/4/28の第3打席の結果」を直接検索するのではなく、「大谷翔平の2023年の全打席結果」などと検索する。

[\[2310.06117\] Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models \(arxiv.org\)](#)

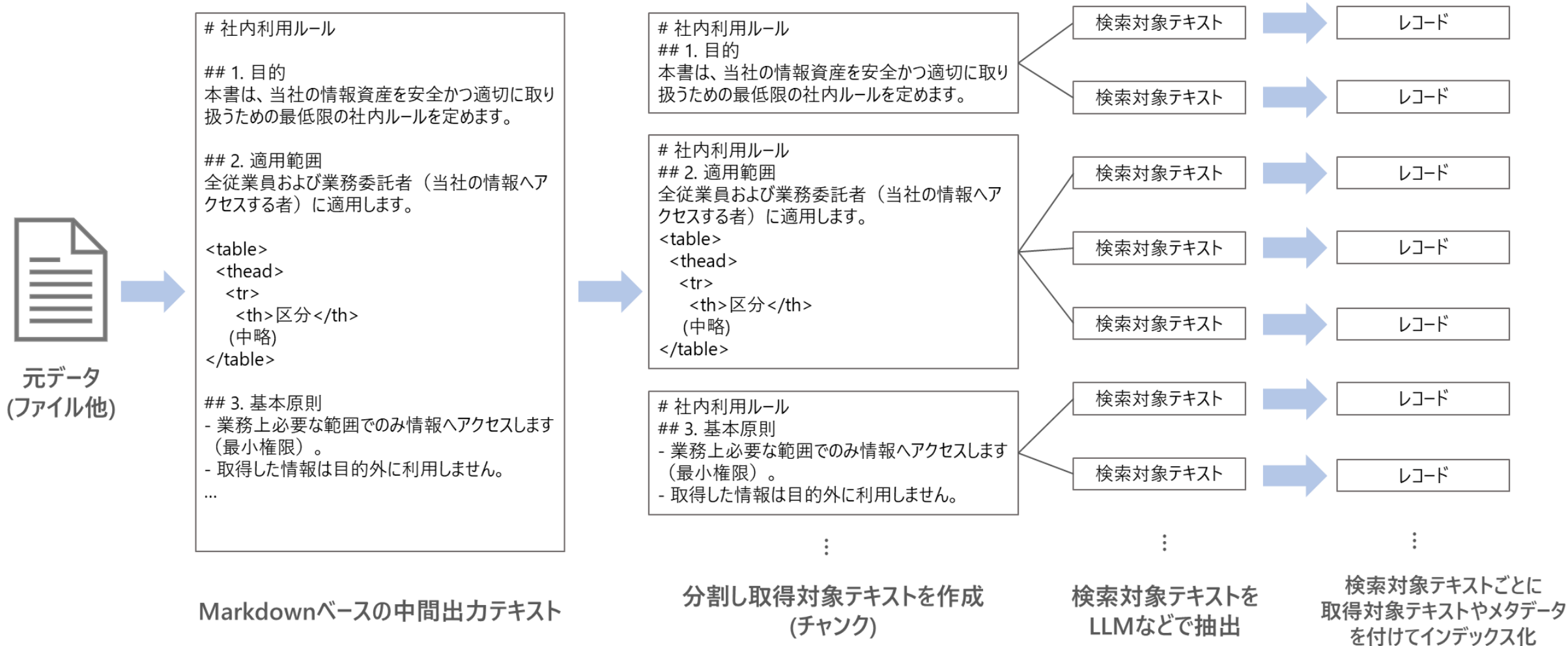
文脈追加



質問に関連する知識生成やFAQ(Shot)の付与。

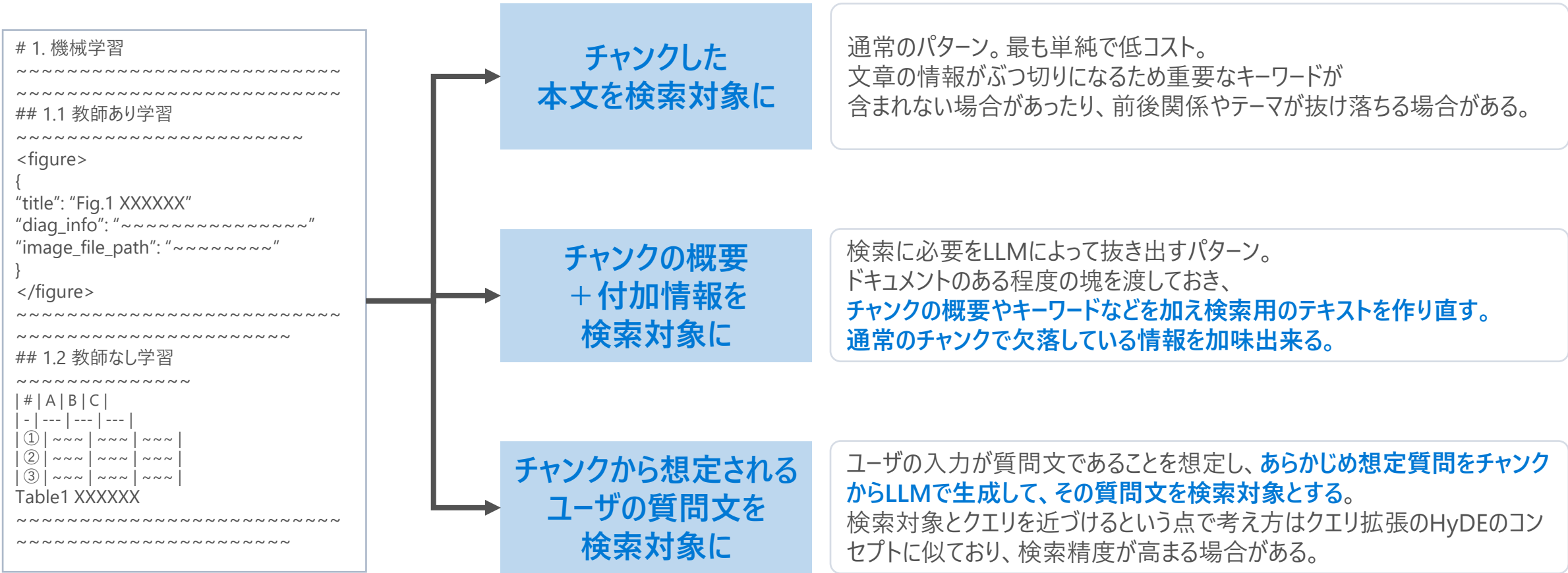
質の良いデータを与えるための抽出パイプラインを把握せよ

LLMへ受け渡すデータをどう作るかは、データに依存するためエンジニアの腕が問われる



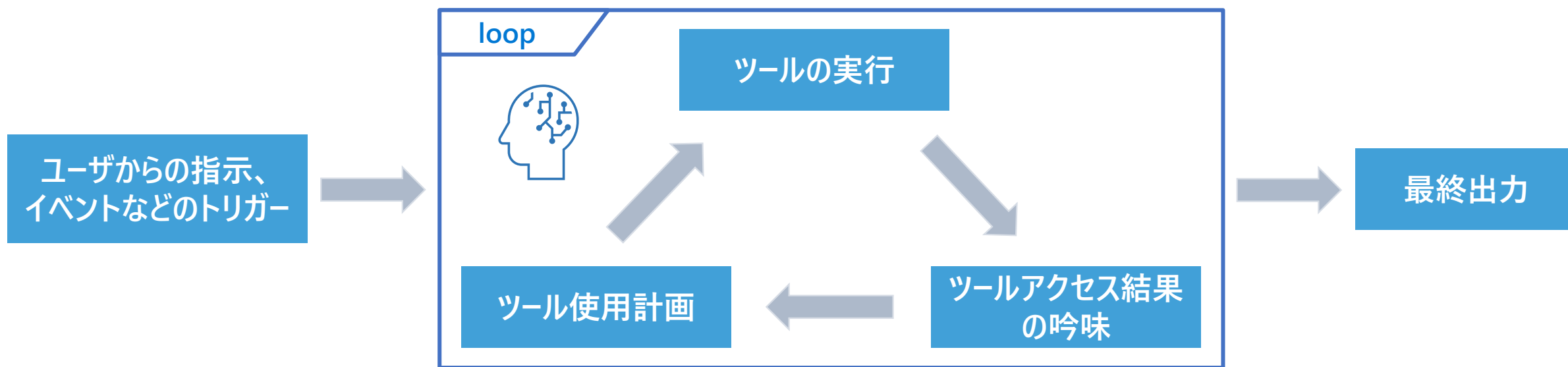
検索対象は必ずしもチャンクした本文ではない

検索対象をチャンクした本文にするという意識が強いが、最終的に渡すテキストと検索対象が同じである必要はない



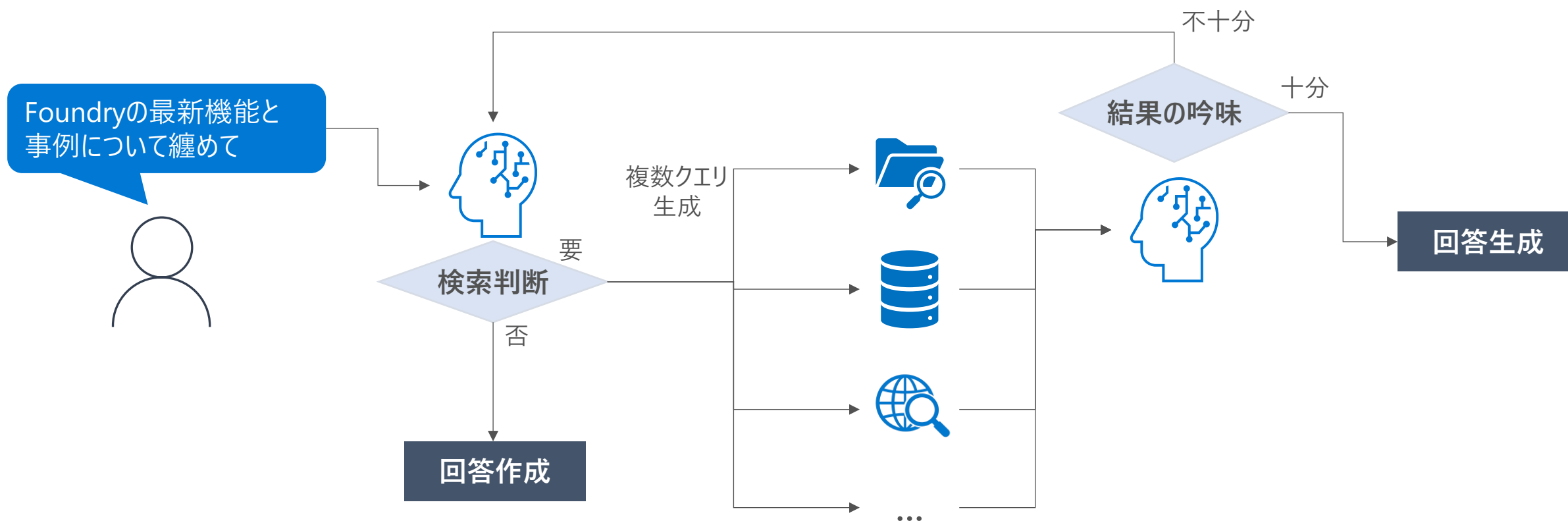
Reasoningモデルの発展と検索への応用

OpenAIのo1から発展した推論モデルが主流になり、この仕組みをRAGに応用。



Agentic RAG の威力と仕組み

推論モデルと組み合わせることでRAGの精度は大幅に向上



Agentic RAG で生じるコンテキスト問題

丁寧で精度的に強力がゆえに Agentic RAG にもいくつかの弊害が存在。



コンテキストの肥大化

検索回数の増加、多様なクエリ、失敗時のリトライ履歴が全てコンテキストに蓄積。扱うトークン量が爆発的に増大し、コストと処理負荷を圧迫する。



検索結果のノイズ化

失敗した検索結果や吟味の過程がコンテキストに残留しノイズとなる。後続のタスク実行時にLLMの性能が低下。



処理の長時間化

検索の失敗や結果の吟味プロセスが発生するたびにLLMとの対話数が増加。ユーザーに対して最終的な回答が返るまでのレイテンシが大幅に悪化する。

Solution

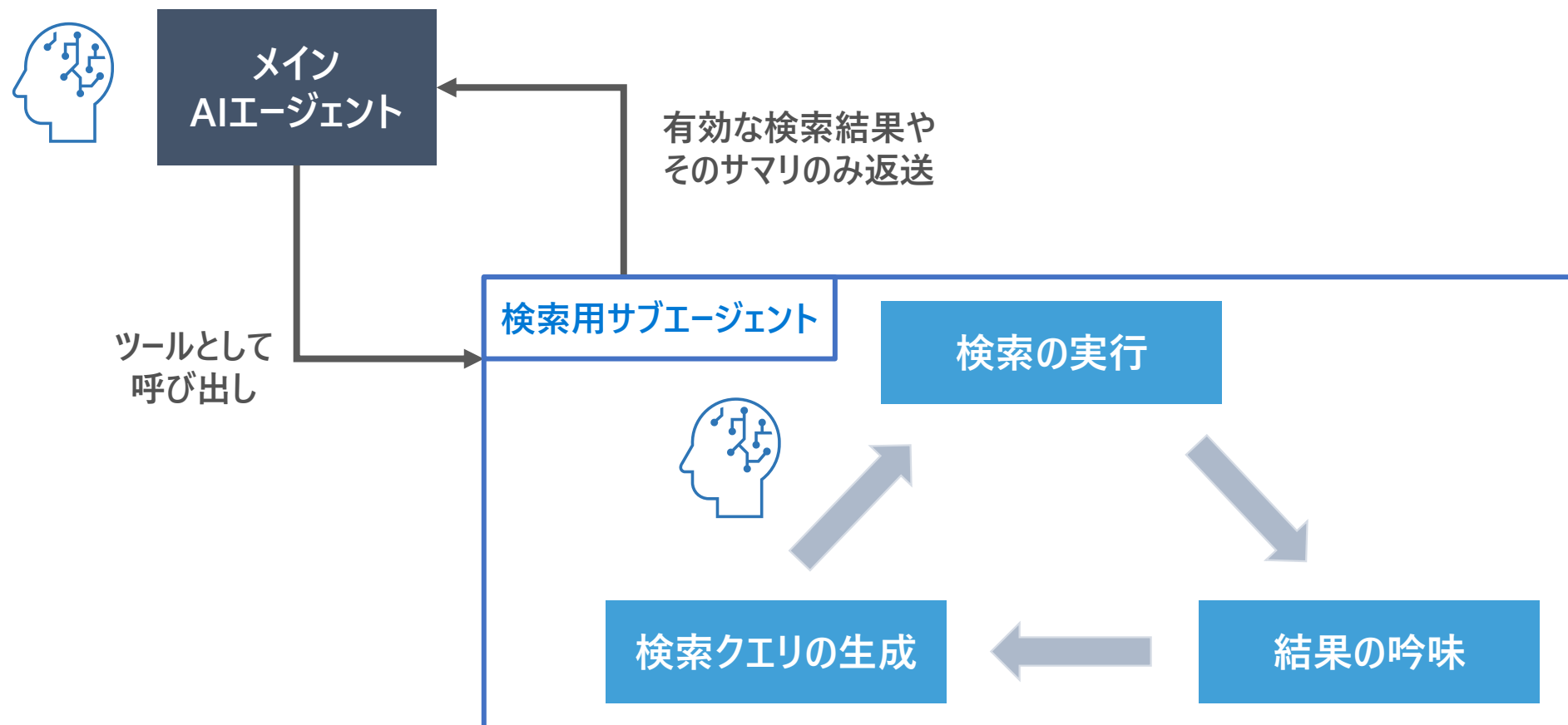
検索のためのAgentを分離する流れが加速 (Agents as Tools)

メインタスク担当と検索担当のエージェントを分離。
モデルやコンテキストを独立させることで、効率的かつ高精度な処理を実現。



Agents as Tools による Agent 分離のイメージ

データを探索するエージェントをメインから独立させることで、Agentic検索を担当するエージェントの最適化とメインコンテキストの圧迫防止が同時に可能





Agent開発で発生するコンテキストエンジニアリング

Agent開発はプロンプトベースで挙動を確認

最も簡単に業務を自動化するには、全てのタスク手順とツール定義・その使い方を全てLLMに持たせて推論モデルで実行してみることが多い。

現代における一般的な初手のエージェント開発

Context

タスクA 指示

ツールA 定義

タスクB 指示

ツールA リファレンス

タスクC 指示

ツールB 定義

タスクD 指示

ツールB リファレンス

⋮

⋮

Shot

出力スキーマ



loop



ツールの実行

ツール使用計画

ツールアクセス結果
の吟味

推論モデルで実行(プロンプトベースエージェント)

複雑化したタスク対処におけるAgentの問題点

Agentをプロンプトのみで実行しようとする、必ず問題が起こる



ツール選択の精度低下

「ツールが増えるほど、判断がブレる」

- ▶ タスク複雑化で適切なツール選択が不安定に
- ▶ 業務ツール・データ過多で候補を絞り切れない
- ▶ 定義を全網羅しても期待精度に届きにくい



手順の誤り

「順序の崩れが、そのまま事故になる」

- ▶ 企業業務は厳密な順序依存（調査→提案→承認）
- ▶ 指示しても、抜け・前倒し・順序違いが発生
- ▶ 特に顧客対応では致命的なミスにつながる



制約の無視

「制約は、プロンプトだけでは守り切れない」

- ▶ 禁止事項、フォーマット、用語統一など多層的制約
- ▶ テキスト指示だけでは抜け落ちる場面が残存
- ▶ 例外処理・厳格運用が必要なほどリスク増大



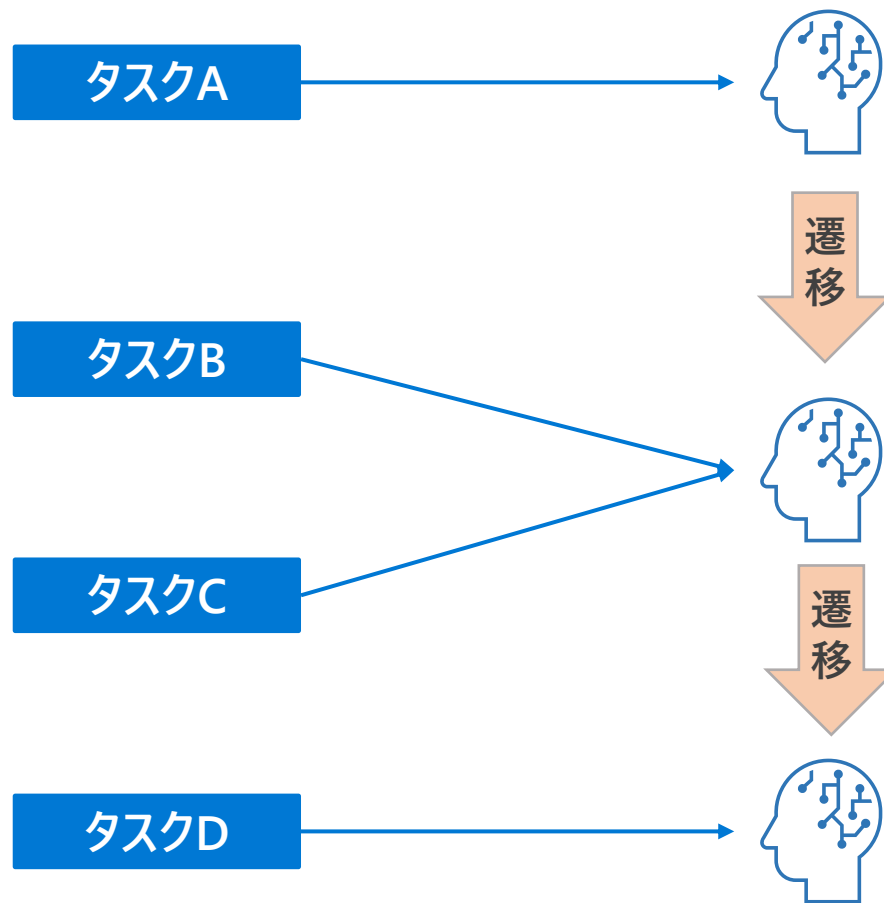
非機能面での問題

「遅い・高いは、それだけで失敗要因」

- ▶ 複雑化に伴い処理時間が伸長、コストも増大
- ▶ 体験品質（UX）と収益性に直結する課題
- ▶ 早期にシステム設計・運用面での対処が必要

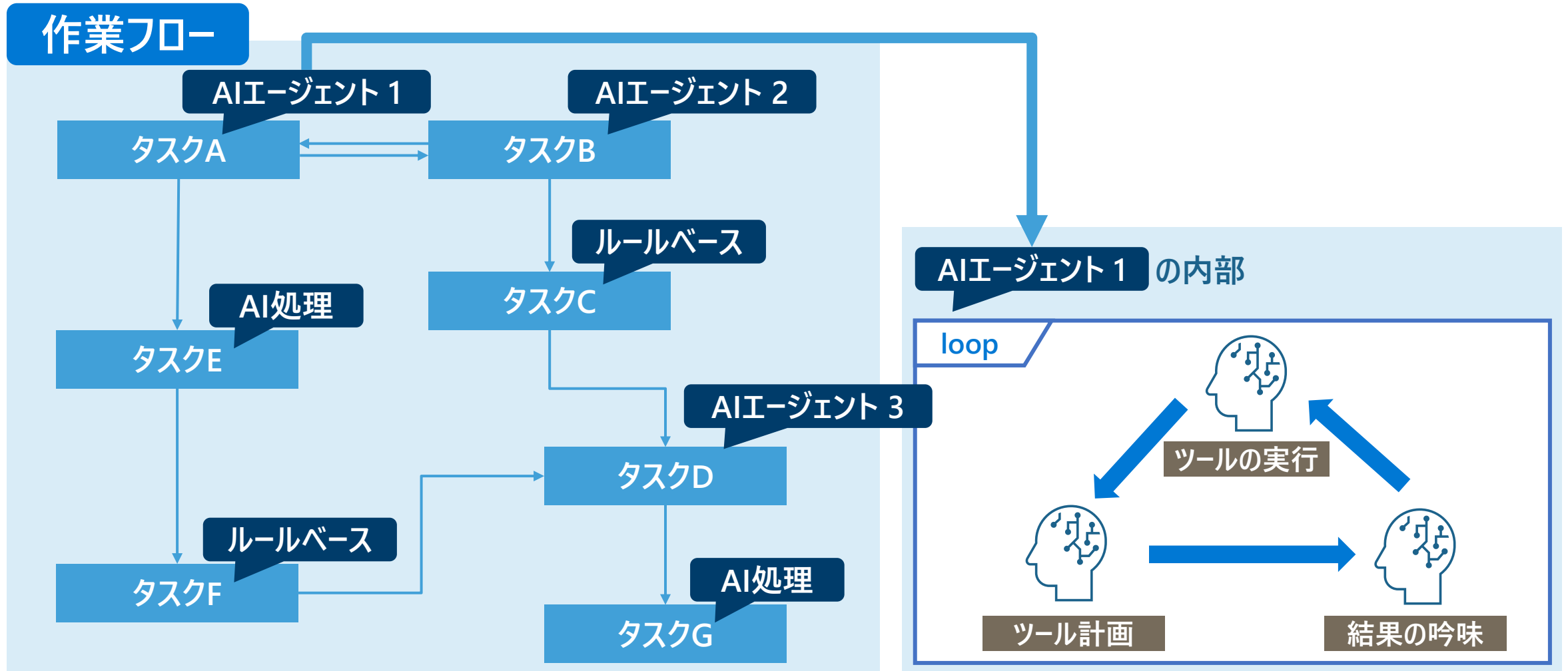
自律性の維持/排除のオーケストレーション (マルチエージェント/ワークフロー化)

最も一般的な対処は、複数のLLMやエージェントにタスクを割り振りワークフロー化すること



最終的な業務自動化システムのイメージ

業務自動化はコンテキストのバランスを見ながら多くの分岐が発生することになる。



MCP隆盛から生まれた「ツール常駐」がボトルネック化

取り込むツールの多さ、タスクの複雑さとAgentへの汎用性の期待でツール定義は増加しがちになる。
また、タスクが汎用になるほど、**コンテキストを圧迫しているのに最終的に使われない**状況も多発。



MCPの普及がユーザによる積極利用を後押し

LLMアプリが外部ツールやデータへ接続するための標準化レイヤが整備。



統合ツールレジストリの肥大化

接続先が増えるほど、**ツール定義 + 補足説明 + 結果**がコンテキストを占有。
LLMへの提示量が増加し、処理コストと遅延が増大。



パフォーマンスへの悪影響

コンテキスト圧迫により、コスト増・レスポンス遅延に加え、LLMの**ツール選択ミス**のリスクが発生。



コンテキスト削減による改善効果

ツール提示を絞るだけで、トークンを**50%超削減**

[RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation](#)

ツール選択精度

13.62% → **43.13%**

汎用CLIツールの人気上昇

特定処理しかできないツールを多く抱えることは効率が悪いので、汎用的な処理ができるCLI環境を用意し、ツールの総量を減らすアプローチが人気に。

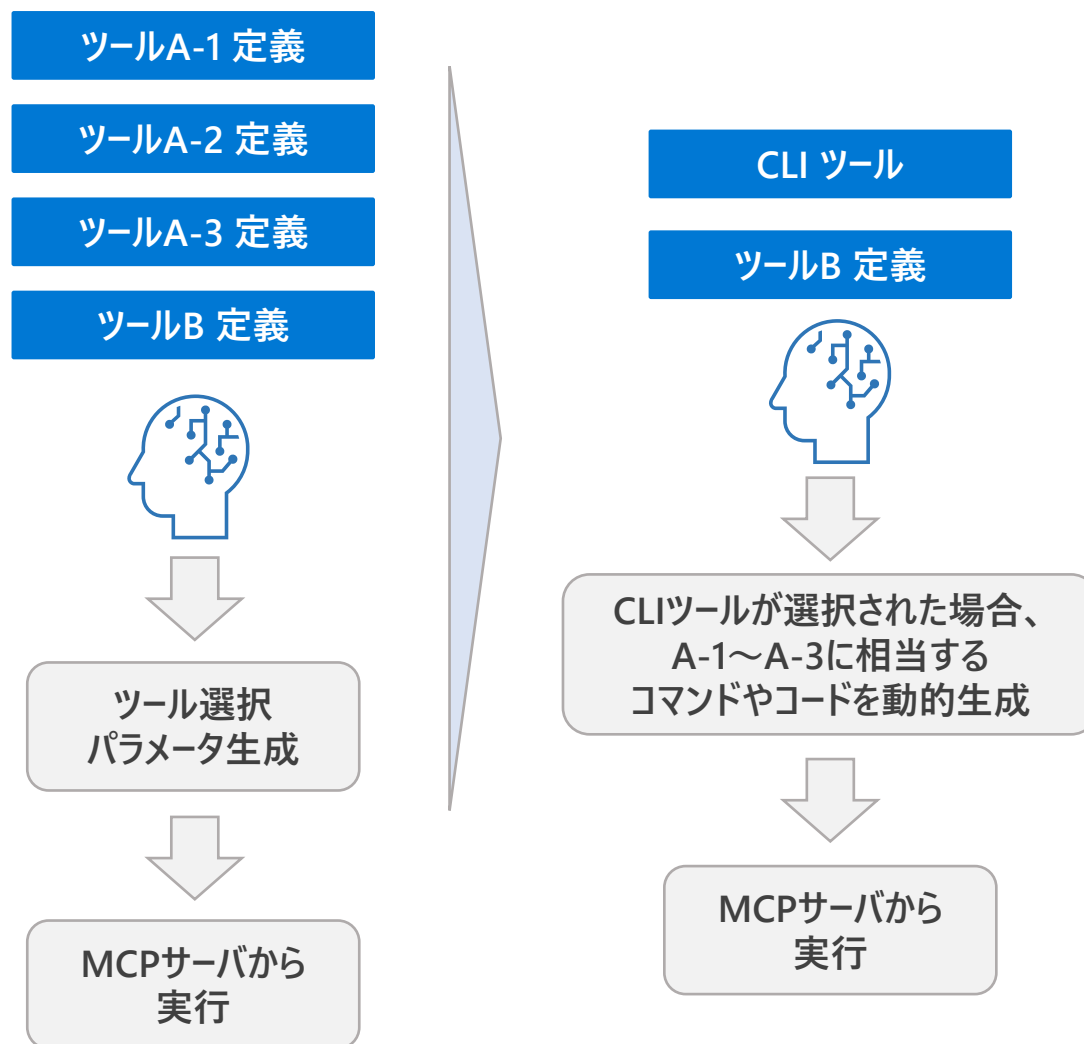
💡 狙いと効果

発想：汎用口への集約

細粒度なツール定義を大量に並べる代わりに、**「汎用の実行口（CLIやコード実行）」**へ機能を寄せる。LLMに提示するツール定義の総量を劇的に減らす。

🔄 メリット・デメリット

多数ツールの定義を常駐させずに済むため、**「多サーバ接続で定義と結果がトークンを食いすぎる」**というボトルネックを緩和。
一方で、**LLMに渡す権限範囲**には注意が必要



Skills による段階的コンテキストロード

単純な処理でなく、関連コンテキストが多いタスクはSkillsとして切り出し。

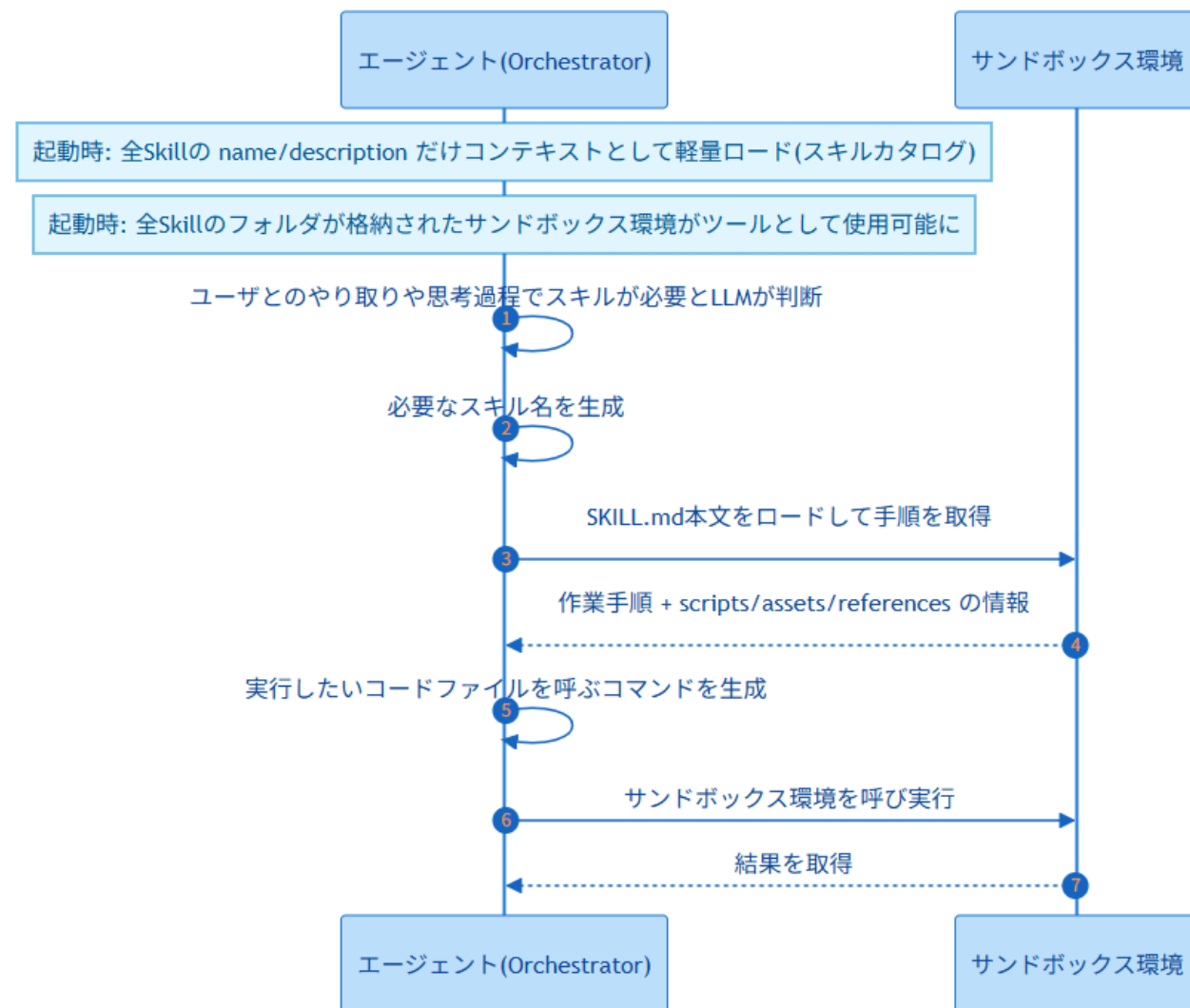
💡 狙いと効果

発想：動的に必要なコンテキストをロード

複雑な手順を伴うサブタスクに関する情報を、
必要なときだけ必要な分のみ段階的にロード。
必要が無ければコンテキストを圧迫せず温存可能。

🔄 メリット・デメリット

「出会う可能性が低いものの、対処は難しくコンテキストは食う」といった手順が存在する場合にコンテキストからの外だしが可能。
Subagentsと比較されることも多い。



Tool Search Tool によるツール定義のRAG

1



ツール情報をインデックス化
使用頻度の大きいツールを
インデックス化

2



検索
手持ちのツールに無いツールが必要
な場合、Tool Search Tool を選択
検索を実行

3



LLM提示 & 実行
取得したツール候補から
LLMがツールを選択し実行

ツールA-1 定義

ツールA-2 定義

ツールA-3 定義

ツールB 定義



使用頻度の低いツールを集約

Tool Search Tool

ツールB 定義



💡 狙いと効果

発想：使用されにくいツールはコンテキストに見せず探させる

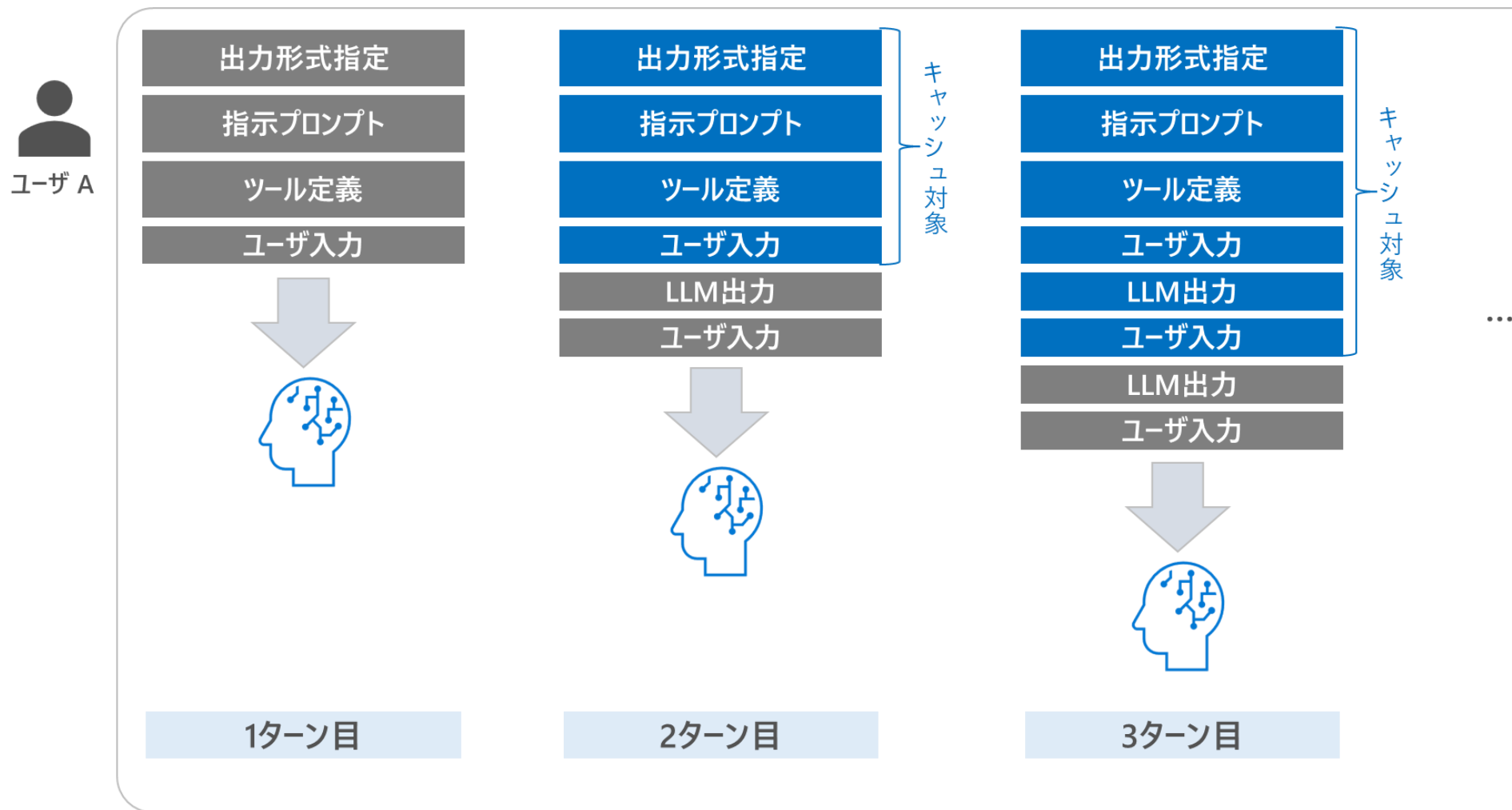
膨大なツール定義を検索エンジン側に寄せることでLLMが処理しなければならないトークンを低減。通常RAGはナレッジを格納するが、これをツール定義に応用。

🔄 メリット・デメリット

検索エンジンにツール定義を寄せられるため、ツール追加に関する精度低下をあまり躊躇する必要がなくなる。

コンテキストキャッシュへの配慮 (単一ユーザ対話継続視点)

コンテキストキャッシュを常に効かせる意識が総コストを大きく抑える。



コンテキストキャッシュへの配慮 (複数ユーザ横断視点)

コンテキストキャッシュを常に効かせる意識が総コストを大きく抑える。

